
Formulaires HTML

Maintenant que vous savez faire des pages et les mettre en forme, il est temps de voir comment poser des questions aux visiteurs de votre site afin, par exemple, de savoir ce qu'ils pensent de votre travail, ou encore de leur permettre de vous envoyer un message de félicitations !

Créer un formulaire

Pour cela, une balise particulière existe

```
<form>
  <!-- Contenu de votre formulaire -->

</form>
```

Cette balise a deux attributs essentiels

- L'attribut `action` définit l'emplacement (une URL) où doivent être envoyées les données collectées par le formulaire.
- L'attribut `method` définit la méthode HTTP utilisée pour envoyer les données (cela peut être « get » ou « post »).
Si on schématise, en GET, les données sont passées comme des paramètres dans l'URL.
En POST, les données ne sont pas directement visibles, elles sont passées dans la requête HTTP.

Ainsi

```
<form action="page.php" method="get">
  <!-- Contenu de votre formulaire -->

</form>
```

va envoyer les données saisies dans le formulaire à la page **page.php** en mettant les valeurs dans les paramètres de l'URL.

Définir des champs

On sait faire un formulaire, maintenant, mais on n'a encore rien demandé comme données au visiteur... Il va falloir proposer des champs à remplir.

Des champs de saisie

Le plus courant est le champ **input** pour entrer quelque chose.

HTML étant un langage de balises, il y a tout naturellement une balise **<input>**. On y reviendra avec les exemples... **input** offre de nombreuses possibilités grâce à l'attribut **type** qui lui permet de définir précisément ce que doit contenir le champ, par exemple... On peut ainsi dire que le champ

input est de type **email**. Cela contraindra l'utilisateur à entrer une adresse mail valide dans le champ. Pour un champ de saisie de texte libre, on mettra simplement le type **text**.

D'autre part, afin d'indiquer à l'utilisateur ce qui est attendu dans le champ (c'est bien la moindre des choses...), il existe la balise **<label>** qui permet de définir une "étiquette". Cette étiquette peut être associée à un champ en passant par son **id** grâce à l'attribut **for**. Un exemple valant mille mots :

```
<!DOCTYPE html>
<html>
  <body>
    <form action="http://1nsi.eurek.ovh/recup.php" method="get">
      <div>
        <label for="name">Nom :</label>
        <input type="text" id="name" name="user_name">
      </div>
      <div>
        <label for="mail">e-mail :</label>
        <input type="email" id="mail" name="user_mail">
      </div>
    </form>
  </body>
</html>
```

les **<div>** encadrant chaque champ ne sont pas du tout obligatoires, mais ça aide à structurer les choses et peut permettre de faciliter la mise en page... A bon entendeur !

Ici, chaque **label** est associé à son **input** par l'intermédiaire de son **id**

```
<label for="name">Nom :</label>
<input type="text" id="name" name="user_name">
```

Entrez le code de l'exemple précédent dans votre éditeur de texte, enregistrez-le et ouvrez la page en double cliquant sur le fichier. Vous devez avoir, dans votre navigateur quelque chose comme :

Nom :

e-mail :

Bien entendu, le CSS vous permettra ensuite de mettre tout cela en forme ! Plus tard...

Il existe beaucoup de types de champs en HTML5. Voyons-en quelques-uns...

- [checkbox](#) : une case à cocher qui permet de sélectionner/désélectionner une valeur
- [date](#) : (HTML5) est un contrôle qui permet de saisir une date (composée d'un jour, d'un mois et d'une année).
- [email](#) : (HTML5) un champ qui permet de saisir une adresse électronique.
- [file](#) : un contrôle qui permet de sélectionner un fichier. L'attribut **accept** définit les types de fichiers qui peuvent être sélectionnés.
- [hidden](#) : un contrôle qui n'est pas affiché mais dont la valeur est envoyée au serveur.
- [number](#) : (HTML5) un contrôle qui permet de saisir un nombre.

- **password** : un champ texte sur une seule ligne dont la valeur est masquée. Les attributs `maxlength` et `minlength` définissent la taille maximale et minimale de la valeur à saisir dans le champ.

Note : Tout formulaire comportant des données sensibles doit être servi via HTTPS. Les navigateurs alertent les utilisateurs lorsque les formulaires avec de telles données sont uniquement disponibles via HTTP.

- **radio** : un bouton radio qui permet de sélectionner une seule valeur parmi un groupe de différentes valeurs.

Le label fonctionne un peu différemment avec les boutons radio... Il contient le libellé du bouton radio...

```
<input type="radio" id="choix1" name="choix" value="Lait">
<label for="choix1">Chocolat au lait</label>

<input type="radio" id="choix2" name="choix" value="Noir">
<label for="choix2">Chocolat noir</label>

<input type="radio" id="choix3" name="choix" value="Blanc">
<label for="choix3">Chocolat blanc</label>
```

produira le résultat suivant

☒ Chocolat au lait ☐ Chocolat noir ☐ Chocolat blanc

L'attribut `value` contient la valeur qui sera passée lors de l'envoi du formulaire.

L'attribut **name** permettra de donner un nom à un champ. C'est ce nom qui sera nécessaire ensuite pour retrouver le contenu du champ en PHP, par exemple...

Ainsi

```
<label for="name">Nom :</label>
<input type="text" id="name" name="user_name">
```

donne le nom **user_name** à ce champ de type **text**.

L'attribut **value** permettra ici de définir une valeur par défaut pour le champ.

l'attribut **placeholder** permettra de donner au visiteur une indication sur ce qui est attendu dans le champ. Le format attendu pour la saisie d'une date, par exemple. Ce n'est pas une valeur par défaut, le champ est vide, mais une indication apparaît en grisé...

```
<label for="date">Date :</label>
<input type="date" id="date" name="date" placeholder="jj/mm/aaaa">
```

Et des boutons

C'est bien joli tout ça, mais même si on a défini où envoyer les données (avec l'attribut **action** de la balise **<form>**), on n'a toujours pas vu comment ajouter un bouton pour les envoyer !

Eh bien, c'est simple, les boutons sont aussi des balises **<input>** avec les types suivants.

- **reset** : un bouton qui réinitialise le contenu du formulaire avec les valeurs par défaut.

- [submit](#) : un bouton qui envoie le formulaire à la page prédéfinie.

Ainsi

```
<input type="submit" value="Envoyer le formulaire">
```

va ajouter ce bouton



Le bouton, une fois cliqué enverra les données saisies par le visiteur à la page indiquée dans l'attribut **action** de la balise **<form>**

Dans l'exemple que vous avez enregistré, la page est <http://1nsi.eurek.ovh/recup.php> c'est une page php que j'ai créée pour vous et qui récupère les données pour les afficher.

Paramètres passés en GET :

```
user_name = Gabriel Gélín  
user_mail = gabriel.gelin@stcharles-stecroix.org
```

Paramètres passés en POST :

Pour définir un bouton, on peut aussi utiliser la balise **<button>**.

```
<button type="submit">Envoyer le message</button>
```

Le principal intérêt de cela est de pouvoir mettre n'importe contenu HTML dans un bouton. une image, par exemple...

```
<button type="submit"></button>
```



En CSS, on pourra rendre tout ça un peu plus joli en retirant la bordure et le fond... Un fichier **style.css** lié à votre formulaire pourrait par exemple contenir les lignes suivantes.

```
button {  
    border: none;  
    background-color: transparent;  
}
```

et on obtiendrait



Zone de texte...

Il existe une autre balise importante pour les formulaires, il s'agit de la balise `<textarea>`. Contrairement à `<input>` ce n'est pas une balise orpheline, elle aura donc une balise fermante.

Ce champ de saisie permet, contrairement à `input` de saisir un texte plus long. `<input>` offre juste une ligne de saisie, `<textarea>` va offrir une zone de saisie qui permettra de saisir plusieurs lignes.

Par exemple :

```
<label for="msg">Message :</label>
<textarea id="msg" name="user_message"></textarea>
```

Comme pour `input`, une valeur par défaut peut être définie pour le texte, mais cette fois, cette valeur sera placée comme contenu de la balise.

```
<label for="msg">Message :</label>
<textarea id="msg" name="user_message">
  Si vous ne changez rien, ce message sera envoyé...
</textarea>
```

provoquera l'affichage suivant :

Message :

Et quelques attributs en plus

Il y a aussi des attributs qui s'appliquent à pratiquement tous les champs. Par exemple, vous avez déjà rencontré des formulaires avec des champs obligatoires. C'est un attribut booléen qui permet cela : ***required***. Sa simple présence rendra le champ où il se trouve obligatoire et le formulaire ne pourra pas être envoyé si le champ n'est pas renseigné.

Par exemple :

```
<label for="name">Nom :</label>
<input type="text" id="name" name="user_name" required>
```

rendra le champ ***user_name*** obligatoire.

Il y en a d'autres que vous trouverez aisément sur internet, mais qui sont sensiblement moins utiles pour un usage quotidien...

Regrouper des champs

On peut définir des groupements de champs, pour ces checkboxes, par exemple, mais pas seulement... On utilisera pour cela la balise `<fieldset>`

```
<fieldset>
  <legend>Veuillez sélectionner vos intérêts :</legend>
  <div>
    <input type="checkbox" id="code" name="interest" value="code">
    <label for="code">Développement</label>
  </div>
  <div>
    <input type="checkbox" id="music" name="interest" value="music">
    <label for="music">Musique</label>
  </div>
  <div>
    <label for="other">Autres :</label>
    <input type="text" id="other" name="other_interest">
  </div>
</fieldset>
```

produira le résultat

Veuillez sélectionner vos intérêts :

☐ Développement

☐ Musique

Autres :

Exercice HTML et CSS

Reproduisez le formulaire suivant :

Nom :

e-mail :

Date :

quelle est votre préférence en matière de chocolat ?

☒ Chocolat au lait ☐ Chocolat noir ☐ Chocolat blanc

Veuillez sélectionner vos intérêts :

☒ Développement

☒ Musique

Autres :

Mot de passe :

Si vous ne changez rien, ce message sera envoyé...

Message :